

DB Summary

Quick reference — query performance, SQL joins, normalization, constraints, relational vs NoSQL, MySQL, and MongoDB.

To Speed Up Query Requests

Technique	What it does
Indexing	Creates a data structure so the DB can locate data without scanning the entire table
Caching	Stores frequently accessed data in memory — fewer disk reads
Query optimization	Better SQL: appropriate join types, fewer subqueries, avoid unnecessary calculations
Partitioning	Splits large tables into smaller pieces — less data scanned per query

Indexing

Technique to speed up data retrieval by creating optimized data structures (B-trees, hash maps, etc.).

Pros

Benefit	Why
Faster query performance	Locate data without full table scans — $O(n) \rightarrow O(\log n)$ or $O(1)$
Efficient sorting and filtering	Optimizes <code>ORDER BY</code> , <code>GROUP BY</code> , and <code>WHERE</code>
Improved JOINS	Foreign key indexes speed up table joins
Unique constraints	Unique indexes (<code>PRIMARY KEY</code>) prevent duplicates
Better full-text search	Specialized indexes (e.g. inverted indexes) enable fast keyword search

Cons

Drawback	Why
Increased storage	Extra disk space — often 10–30% of table size
Slower writes	<code>INSERT</code> / <code>UPDATE</code> / <code>DELETE</code> must update indexes too
Maintenance overhead	Indexes fragment — periodic <code>REINDEX</code> or <code>OPTIMIZE TABLE</code>
Over-indexing	Unused indexes waste resources; too many slow write-heavy systems
Not all queries benefit	Useless for <code>LIKE '%term%'</code> (leading wildcards) or low-cardinality columns (e.g. gender M/F)

Use cases

- High-read, low-write systems (e.g. reporting databases)
- Columns frequently used in `WHERE` , `JOIN` , or `ORDER BY`
- Primary / foreign keys (almost always indexed)

Avoid

- Write-heavy tables (e.g. logging systems)
- Columns rarely queried or updated
- Small tables (full scans may beat index lookups)

Index types

Type	Best for	Example
B-tree	Range queries, sorting	<code>WHERE age > 25</code>
Hash	Exact matches (no ranges)	<code>WHERE id = 123</code>
Full-text	Keyword search	<code>WHERE text LIKE '%database%'</code>
Composite	Multi-column queries	<code>WHERE city = 'NY' AND age > 30</code>

Partitioning

Split large tables into smaller **partitions** while treating them as one logical table. Widely used in databases for scale and maintenance.

Pros

Benefit	Why
Improved query performance	Scan only relevant partitions, not the full dataset (e.g. date filter skips old partitions)
Efficient maintenance	Drop whole partitions vs row-by-row delete — <code>ALTER TABLE logs DROP PARTITION p_2022</code> vs <code>DELETE WHERE year = 2022</code>
Cost savings	In cloud DBs, querying fewer partitions reduces compute cost
Scalability	Spread data across disks/nodes to reduce bottlenecks

Cons

Drawback	Why
Complexity	Needs careful partition key choice; wrong key can hurt performance
Limited joins	Cross-partition joins slower unless partition keys align
Overhead	Partition metadata uses memory; cross-partition queries still cost
Not for small tables	Unnecessary complexity when data volume is low

Types

- **Range** — by value ranges (e.g. dates)
- **List** — by explicit value lists (e.g. regions)
- **Hash** — even distribution by hash of key
- **Key** — similar to hash, DB-managed hashing

Use cases

- Large tables (>10 GB in SQL DBs; >1 TB in distributed systems)
 - Time-series data (logs, IoT sensor data)
 - Frequent range queries (e.g. `WHERE date BETWEEN ...`)
-

Join Types

Join	Description
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All rows from left table + matches from right
RIGHT JOIN	All rows from right table + matches from left
FULL JOIN	All rows when there's a match in either table
SELF JOIN	Joins a table to itself

Performance considerations

- Always join on **indexed columns**
 - Use **table aliases** for complex queries
 - Be specific about columns — avoid `SELECT *`
 - Use **EXPLAIN** to analyze join performance
-

WHERE vs HAVING

	WHERE	HAVING
When	Filters rows before grouping	Filters groups after aggregation
Used with	Regular columns (non-aggregate)	Aggregate functions (<code>SUM</code> , <code>COUNT</code> , <code>AVG</code> , etc.)
Efficiency	More efficient — reduces data before grouping	Less efficient — filters after grouping
Statements	<code>SELECT</code> , <code>UPDATE</code> , <code>DELETE</code>	<code>SELECT</code> with <code>GROUP BY</code> only

Relational Databases

Fixed schema — data stored in **tables** with predefined columns and data types. Rows hold data; **foreign keys** link tables.

Good for:

- Transactional systems — data integrity and consistency matter
- ACID guarantees (all operations succeed or all roll back)
- Minimizing redundancy, efficient querying and indexing

Characteristics:

- Mature, stable model
- SQL for querying and manipulating data
- Schema-on-write — each table has a predefined structure

Limitations:

- Less flexible when data structure changes frequently
- Not ideal for highly dynamic or unstructured data models

Primary key vs foreign key

	Primary key	Foreign key
Role	Uniquely identifies each row in a table	Links two tables — references another table's primary key

Normalization

Break a large table into **smaller tables** and link them with **primary** and **foreign keys**.

- Eliminates **redundant data** — each fact stored in one place
- Reduces **data anomalies**
- Improves **data integrity**

Core idea for relational databases: structure data so updates, inserts, and deletes stay consistent across related tables.

Schema Constraints

Rules that define the structure and properties of a database schema. Applied to tables, columns, or relationships — data must conform to these rules.

Good for relational databases — ensure integrity and consistency; prevent duplicate data, orphaned records, and referential integrity violations.

Constraint	Purpose
Primary key	Each row uniquely identified by a column (or set of columns). No duplicate rows; efficient lookups by key.
Foreign key	Values in one table must match a primary key in another table. Enforces referential integrity between related tables.
NOT NULL	Column cannot contain null values. Prevents missing required data.
UNIQUE	Values in a column (or set) must be unique. Prevents duplicate data.
CHECK	Custom rule on allowed values (e.g. <code>age > 0</code>). Prevents invalid data.

Choose constraints carefully for each schema so data stays properly structured and maintained.

MySQL

Relational database — widely used for web apps, e-commerce, and transactional systems.

Good at:

- ACID transactions — data integrity and consistency
- Structured data with fixed schema
- SQL-based reporting and multi-table logic

Bad at:

- Rapidly changing or unstructured data shapes
 - Massive horizontal scale without careful architecture
 - Workloads that fit specialized NoSQL models better
-

MySQL vs PostgreSQL

Both are open-source relational databases that support SQL, ACID transactions, indexes, replication, stored procedures, and JSON.

	MySQL	PostgreSQL
Focus	Simplicity and common web workloads	Correctness, extensibility, and complex workloads
SQL compliance	Supports core SQL with some MySQL-specific behavior	More closely follows SQL standards
Data types	Strong support for common relational types and JSON	Rich types including arrays, ranges, <code>JSONB</code> , and custom types
Complex queries	Good for straightforward read-heavy and transactional workloads	Strong for complex queries, analytics, and concurrent writes
Extensibility	Plugins and configurable storage engines	Custom types, operators, functions, and extensions such as PostGIS

	MySQL	PostgreSQL
Full outer join	No native <code>FULL OUTER JOIN</code>	Native <code>FULL OUTER JOIN</code> support
Typical use	Web apps, e-commerce, and systems already using the MySQL ecosystem	Data-heavy applications, geospatial systems, analytics, and complex business logic

General choice: PostgreSQL is often the stronger default for new applications because of its richer features and stricter data handling. MySQL remains a good choice when the team, hosting, or existing infrastructure already relies on its ecosystem.

Non-Relational (NoSQL) Databases

No fixed schema — flexible data models that adapt to changing structures.

Good for:

- Large volumes of **unstructured or semi-structured** data (social posts, sensor data, logs)
- High scalability and performance
- Rapid development and iteration
- Diverse data types and structures

Challenges:

- Data consistency and validation are harder — choose model and tech carefully
- Duplication is often allowed; strict schema constraints are not enforced

NoSQL data models

Model	Description	Use cases
Document	JSON-like nested documents with varying structure	Flexible catalogs, content
Key-value	Simple key → value storage	Caching, sessions, simple lookups
Column-family	Wide columns, high write throughput	Large-scale analytics, time-series
Graph	Nodes and edges for relationships	Social networks, recommendations, fraud detection

MongoDB

Document-based NoSQL database.

Good at:

- JSON-like **documents** — nested, varying structures
- **Dynamic schema** — easy model changes
- **Sharding and replication** — high availability, scalability, fault tolerance

- Unstructured / semi-structured data at scale

Bad at:

- Complex multi-table joins and strict relational integrity
 - Heavy transactional workloads needing strong ACID across many entities
 - Fixed-schema reporting that fits SQL better
-

Summary

	Relational (e.g. MySQL)	NoSQL (e.g. MongoDB)
Best fit	Transactional systems needing consistency and integrity	Large unstructured/semi-structured data, scale, flexibility
Schema	Fixed, predefined	Flexible, dynamic
Strength	ACID, SQL, relationships via keys	Scale, speed, adaptable data models

Relational databases suit transactional systems that require data consistency and integrity. NoSQL databases suit large volumes of unstructured or semi-structured data and goals like high scalability and performance.